

01_Apuntes_Seaborn_Librerias_Visualizacion_Datos

May 11, 2026

1 Introduccion a librerias de visualizacion de datos

1.1 Contexto de los visualizadores de datos

Hasta ahora hemos aprendido en estas unidades a conocer y utilizar el lenguaje Python, a grandes rasgos y posteriormente la gestion de datos aplicada, a traves de las librerias de calculo cientifico Numpy y Pandas.

Ahora mismo, disponemos de las herramientas necesarias para trabajar con ficheros con datos, crear objetos o no, integrar los datos en el programa y modificar o realizar calculos en tablas.

El siguiente paso es preparar **la visualizacion de estos calculos** o datos que hemos transformado en nuestra fase previa.

Para ello, existen librerias en Python especializadas en la generacion de graficos de datos.

Nos centraremos en *Seaborn*

Estas son:

- ***Matplotlib***: Es la libreria mas extendida de visualizacion de datos en Python. Es la base de *Seaborn*
- ***Seaborn***: Dedicaremos la unidad a esta libreria. Amplia las opciones de *Matplotlib*
- ***ggplot***: Libreria inspirada en *ggplot2* de R. Basada en *Matplotlib* e integrada en *Pandas*
- ***Plot.ly***: Libreria dedicada a generar y compartir graficos online.

Como veis, *Matplotlib* es la base de estas funcionalidades en Python, sin embargo, ha quedado algo obsoleta en comparacion a otras que han facilitado su uso, como *Seaborn*.

Para poder hacer uso de estas funcionalidades de visualizacion en Jupyter o Spyder, siempre deberemos utilizar la siguiente sentencia:

%matplotlib inline

Sin no ejecutamos esto no se podran insertar los graficos entro del documento, no es codigo python y no debe aparecer en los scripts, es unicamente para Jupyter

En caso de que trabajemos en **otros entornos**, sera necesario usar la siguiente sentencia:

import matplotlib.pyplot as plt

```
[1]: %matplotlib inline
```

1.2 Iniciacion a Seaborn: Definicion

Es una libreria open source basada en *Matplotlib*, que permite una facil integracion con Pandas y sus DataFrames.

La idea de esta libreria es **enriquecer el trabajo con los datos en python, dando opciones de visualizacion** en el IDE.

Seaborn tiene algunas características como libreria que es necesario indicar:

- La gran mayoría de funcionalidades para personalizar visualizaciones están disponibles usando *Seaborn*, **pero puede ser necesario utilizar *Matplotlib* en algún momento dado para algunas personalizaciones**
- Al cargar *Seaborn*, obligatoriamente también cargas, *numpy*, *scipy*, *pandas* y *matplotlib*
- Opcionalmente puedes cargar *statsmodel* **para modelos de regresión completos** y *fast-cluster* **para realizar clustering**

Para instalarlo, utilizamos *pip install* como en otras librerías

pip install seaborn

```
[2]: pip install seaborn
```

```
Requirement already satisfied: seaborn in c:\users\insau\anaconda3\lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\insau\anaconda3\lib\site-packages (from seaborn) (2.1.3)
Requirement already satisfied: pandas>=1.2 in c:\users\insau\anaconda3\lib\site-packages (from seaborn) (2.2.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\insau\anaconda3\lib\site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\insau\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.0)
```

```
Requirement already satisfied: python-dateutil>=2.7 in
c:\users\insau\anaconda3\lib\site-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
c:\users\insau\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
c:\users\insau\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\insau\anaconda3\lib\site-
packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

1.3 Funciones disponibles en Seaborn: Modulos y Estructuras

Una vez tenemos instalado Seaborn, lo siguiente sera traerlo a nuestro script para poder utilizarlo.

Primero usaremos esta sentencia para traer la libreria y sus metodos al script:

```
import seaborn as sns
```

Importante: Todas las sentencias de esta libreria estan en **el nivel mas alto (o nivel raiz)** por lo que para llamar a sus metodos, solo sera necesario utilizar *sns.***“nombre del metodo”**

Para entender las funcionalidades de la libreria es necesario entender que existen 2 características para explicar los graficos

Estos son los Modulos y las Estructuras.

TIP IMPORTANTE: Para que se pueda visualizar el grafico, es necesario hacer uso de una sentencia o de “;” despues de haber escrito el grafico

1º opcion: *plt.show()*

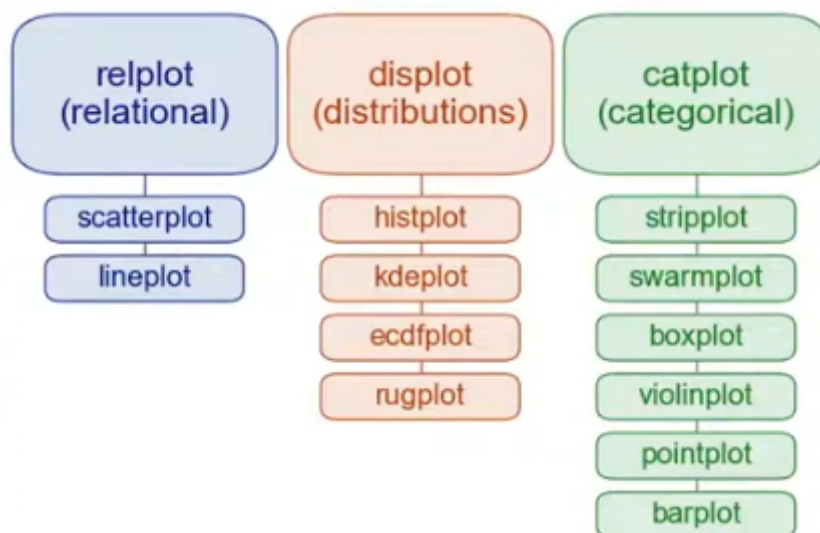
2º opcion: *sns.***“metodo para crear grafico”(....) ;**

1.3.1 Modulos de Seaborn

Los modulos se pueden entender **la forma en la que se van a visualizar los datos**

Como norma general, **los metodos que compartan el mismo modulo, compartiran características similares** es decir, podremos realizar diferentes tipos de visualizaciones respetando el modo de representacion (pasar de un grafico a otro, en esencia).

Los modulos disponibles que agrupan los distintos tipos de graficos seran los siguientes:



En resumen existen estos 3 grupos:

- **Modulo Relacional:** Nos permiten ver interacciones o dependencias entre 2 variables
- **Modulo de Distribucion:** Nos permiten visualizar distintas medidas descriptivas de la distribucion de los datos (media, minimos, maximos...)
- **Modulo de Categorias:** Nos permiten ver como varia los datos pertenecientes a una variable en funcion del nivel o categoria, (otra variable.)

Existe otro grupo mas denominado **FacetGrid** que se considera las figuras/objetos de mas alto nivel de la libreria, nos permitira **ver relaciones o dependencias entre mas de 2 variables**

1.3.2 Estructuras de Seaborn

Las estructuras, se podrian entender como **el como se generan las visualizaciones de datos**

Seaborn permite generar graficos **a partir de datos organizados en vectores**, diferenciando entre 2 tipos de estructuras:

- **Long-form data**
- **Wide-form data**

Esta diferenciacion es crucial, ya que segun el tipo de dato, **Seaborn los tratara diferente y obtendremos distintos resultados**

Long-form Data Se podran considerar de este tipo de estructura de dato si tiene estas caracteristicas:

- Cada **variable es una columna** > Ejemplo: *caracteristicas definitorias de una persona (edad, altura,...)*
- Cada **observacion es una fila** > Ejemplo: *persona 1, persona 2,...*

Con estas estructuras **podemos especificar explicitamente las columnas a las variables que deseemos**, por lo que podremos **acomodar sets de datos de alta complejidad siempre que nuestras variables esten claramente definidas**

En definitiva, nos permitira trabajar con cualquier tipo de dato, independientemente de la complejidad del mismo, ya que **tendremos el control sobre que variables mostrar o que observaciones**

Una sentencia tipo de esta estructura es asi:

```
sns.relplot ( data = "nombre dataframe" , x = "nombre columna", y = "nombre columna", hue = "nombre columna", kind = "tipo de elemento grafico")
```

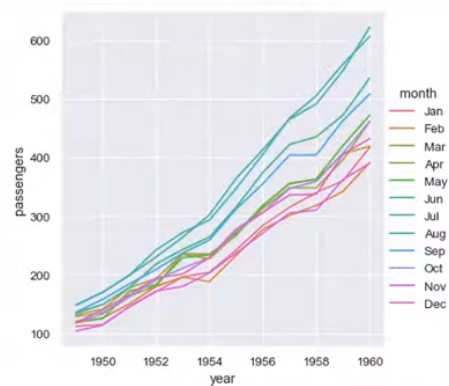
Este ejemplo tiene estas características:

El grafico sera del **Modulo Relacional**

Se agrupara la informacion en base a otra columna por el uso de **hue** (un *groupby* de pandas)

El elemento grafico (lineas, puntos,...) se definira mediante **kind**

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121



Wide-form Data Son datos tabulares que disponen de una característica distinta, **ya que cada columna/fila contiene niveles de diferentes variables**

Como veíamos antes, generalmente, en cada fila se encuentra un dato definido por la variable de la columna

Ejemplo: La fila puede ser el registro y en cada columna se identifica, una variable distinta del registro.

Por tanto en estas estructuras **cada celda se encuentra definida por las filas y columnas que son sus variables**

Ejemplo: la fila puede ser *el año* y la columna puede ser *el mes* y el dato de la celda puede ser *el n° de elementos*

Por lo que cada celda se entiende gracias a las coordenadas de las filas y las columnas, que actúan como índice

Por lo que a la hora de crear la grafica no podremos acceder a las variables por su nombre, sino que **seaborn buscara la relacion fila/columna para entender el dato**

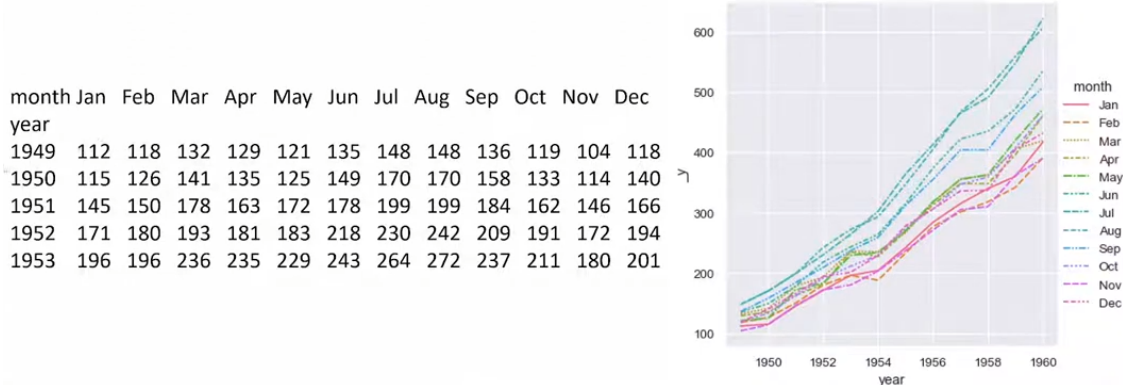
Una sentencia tipo de esta estructura es asi:

```
***sns.relplot ( data = “nombre dataframe”_wide , kind = “tipo de elemento grafico”)**
```

Este ejemplo tiene estas características:

El grafico sera del **Modulo Relacional**

El elemento grafico (lineas, puntos,...) se definira mediante **kind**



Diferencias entre *Long-form* y *Wide-form* Vistas las estructuras, ahora podemos extraer algunas conclusiones importantes para usarlas:

- 1º: Si tratamos los datos como *Long-form* **ganaremos mas control y complejidad sobre la representacion** mientras que en los *Wide-form* **simplificamos el uso de los datos y su representacion, perdiendo cierta informacion, en concreto, etiquetas que especificas que representan al dato**

En resumen, perdemos informacion del eje Y en el grafico al pasar de *Long* a *Wide*

- 2º: **Hace falta menos parametros para crear el grafico utilizando estructuras *Wide-form***
- 3º: Si los datos estan estructurados en *long-form*, **podemos diseñar graficos distintos sin cambiar la estructura**

1.3.3 Facets

Hasta ahora, hemos visto que Seaborn nos va a permitir **construir una visualizacion con una linea de codigo**, pero hay situaciones en las que necesitamos disponer de mayor nivel de personalizacion, para entender los datos con los que tarbajamos y esto va a requerir de usar mas de una linea de codigo.

Estas visualizaciones mas complejas requieren hacer uso de los **FacetGrid**, que nos va a permitir, **mostrar varias visualizaciones al mismo tiempo**

Su sintaxis se compone de los siguientes parametros:

- **data:** Sera el dataframe *long-form* donde cada columna es una variable y cada fila una observacion

- **col, row:** Ayudaran a definir los subsets, cada una se mostrara en graficas diferentes. > **col:** Define el nº de graficos por columnas > > **row:** Define el nº de graficos por filas
- **hue:** Nos permite agrupar datos por una variable concreta
- **x,y:** Seran los ejes de los graficos, basadas en columnas del dataframe

Haremos uso de las siguientes sentencias para poder crear los graficos:

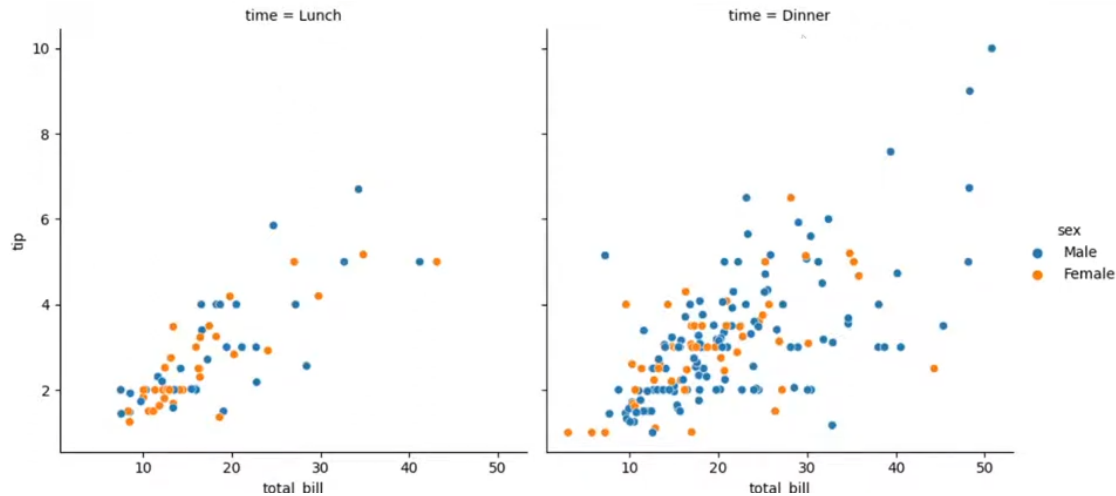
“nombre dataset para usar en el grafico” = sns.load:dataset (“nombre dataset”)

sns.“modulo grafico”(x = “nombre columna”, y = “nombre columna”, hue = “ nombre columna que agrupara“, col =”nombre columna que separa los graficos en funcion de los distintos valores“, data =”nombre dataset para usar en el grafico”)

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Lunch	3
.....							
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
sns.relplot(x="total_bill", y="tip", hue="sex",
            col="time", data=tips)
```



Podemos realizar una complejidad mayor, en la que **queramos filtrar ciertos datos para los distintos graficos**, en ese caso utilizaremos las sentencias:

“nombre dataset para usar en el grafico” = sns.load:dataset (“nombre dataset”)

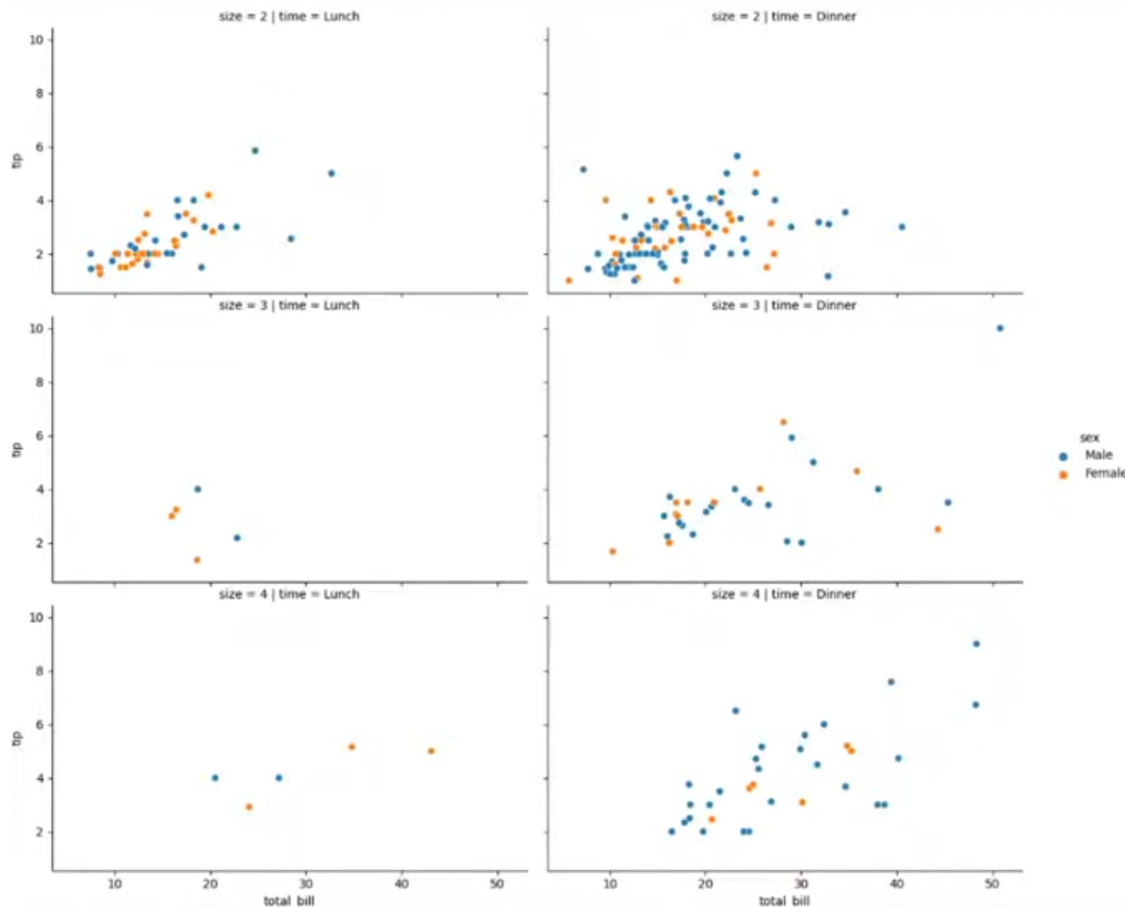
“nombre dataset para usar en el grafico” = “nombre dataset anterior”.loc [(“nombre dataset anterior”[“dimension por la que filtrar”] > “valor”) & ...]

sns. “modulo grafico”(x = “nombre columna”, y = “nombre columna”, hue = “ nombre columna que agrupara“, col = “nombre columna que separa los graficos en funcion de los distintos valores“, row = “nombre dimension por la filtrar“, data = “nombre dataset para usar en el grafico”)

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Lunch	3
.....							
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2


```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
tips = tips.loc[(tips["size"] > 1) & (tips["size"] < 5)]
sns.relplot(x="total_bill", y="tip", hue="sex",
            col="time", row="size", data=tips)
```



1.4 Diferentes tipos de graficos por modulos y *Facets*

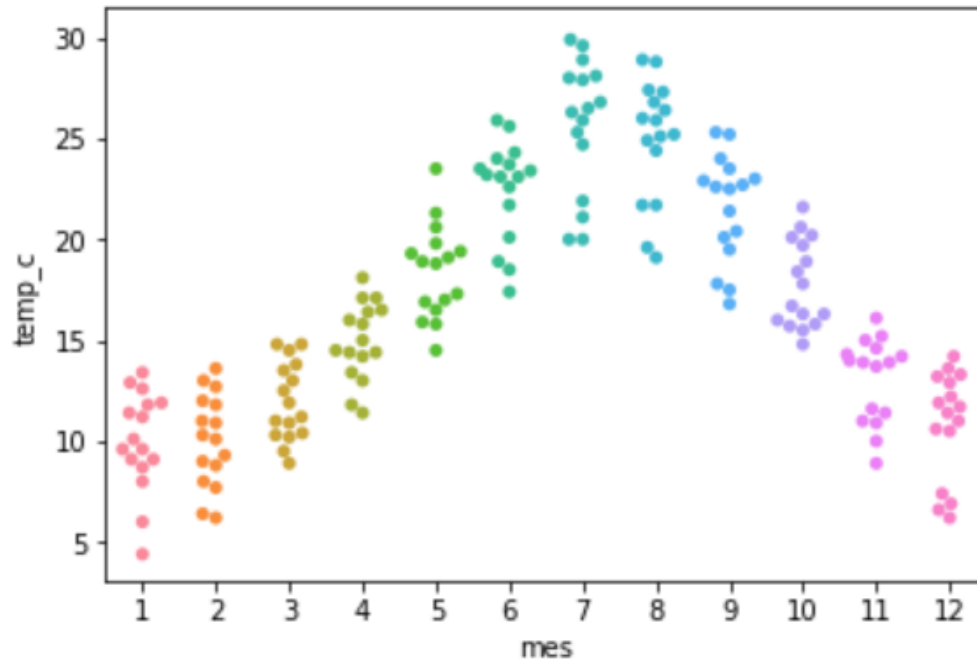
A continuacion veremos algunos metodos que nos van a permitir generar diferentes tipos de graficos disponibles en los distintos modulos de Seaborn

sns.swarmplot()

Grafico de categoria Este metodo generara un grafico de dispersion evitando que se solapen los puntos, donde se representan las observaciones de 1 variable en funcion de una categoria (otra variable)

sns.swarmplot (*data* = “nombre dataframe”, *x* = “nombre columna”, *y* = “nombre columna”)

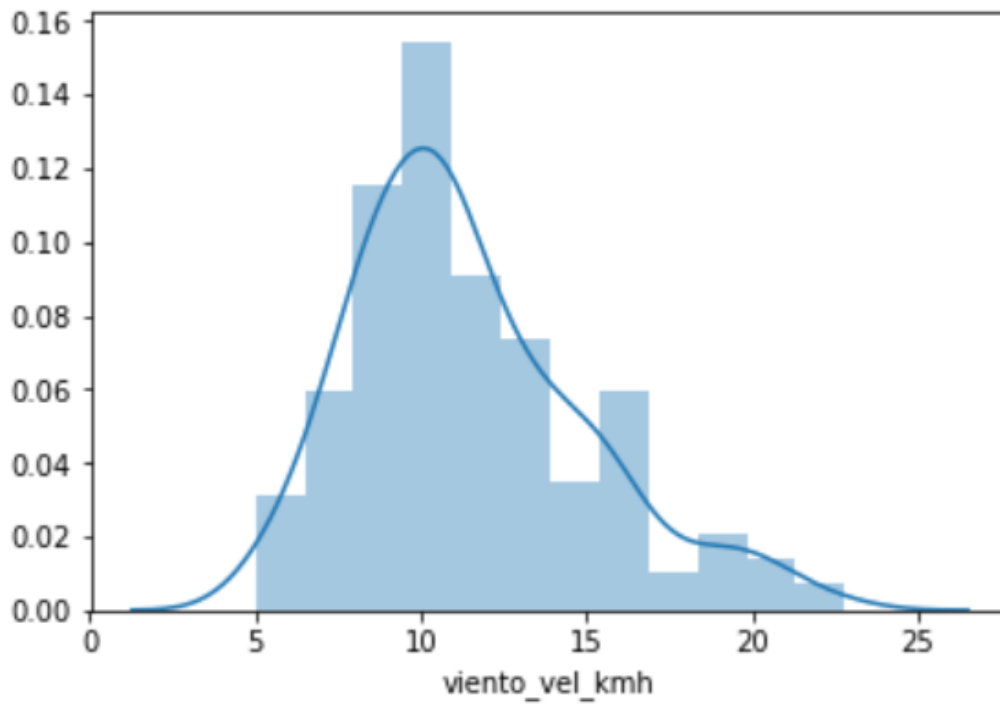
Se puede incluir un agrupador mediante el parametro *hue*



sns.distplot()

Grafico de distribucion Este metodo generara un histograma y una estimacion de la densidad de la distribucion de los datos

sns.distplot (“nombre dataframe”[“columna o vairable por la que ver la distribucion”])



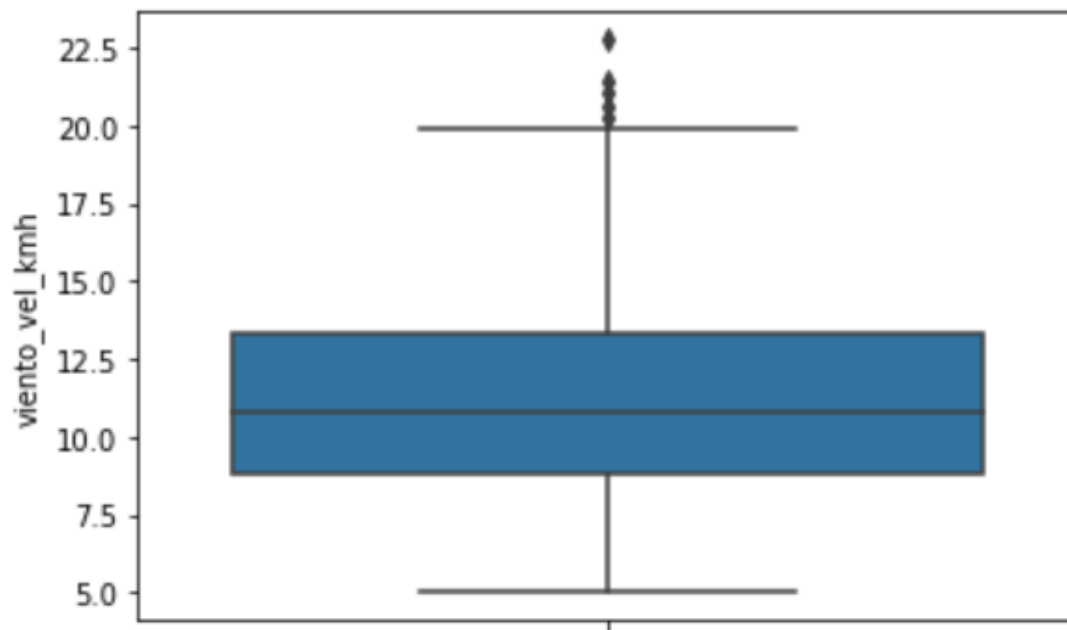
Permite añadir 2 parametros adicionales para personalizarlo como:

- *kde = False* : Permite omitir el estimador de densidad
- *rug = True*: Permite visualizar el nº de observaciones en el eje X

sns.boxplot()

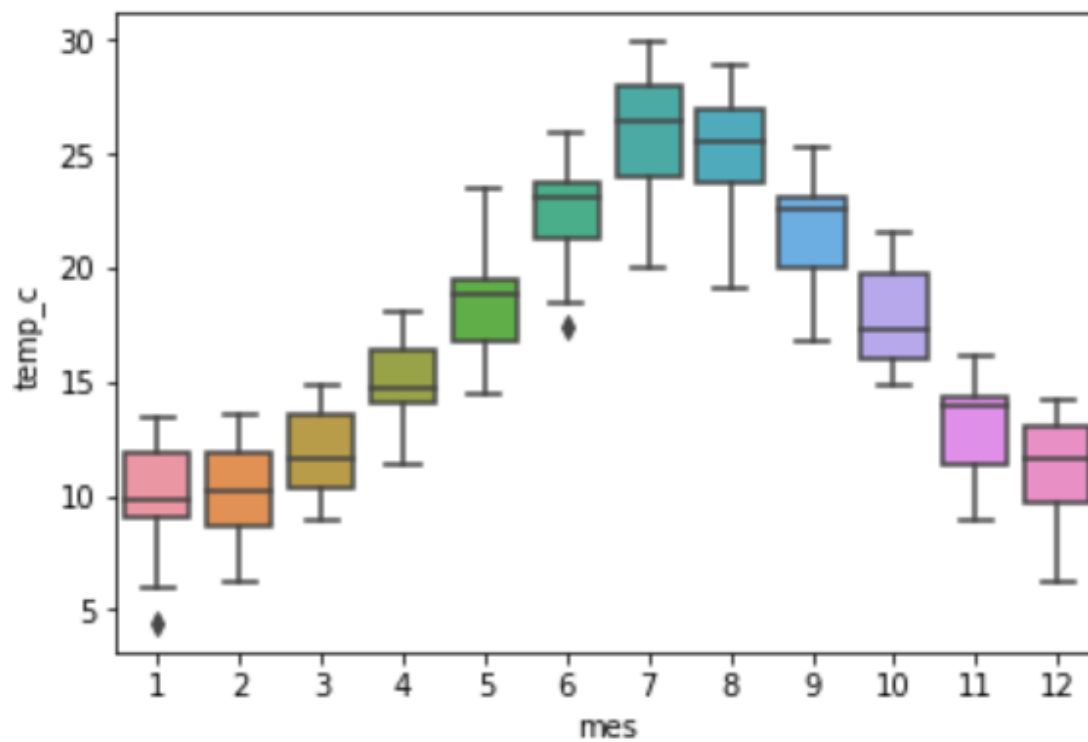
Grafico de categoria Este metodo generara un diagrama de caja

sns.boxplot (data = “nombre dataframe”, y = “nombre columna”)



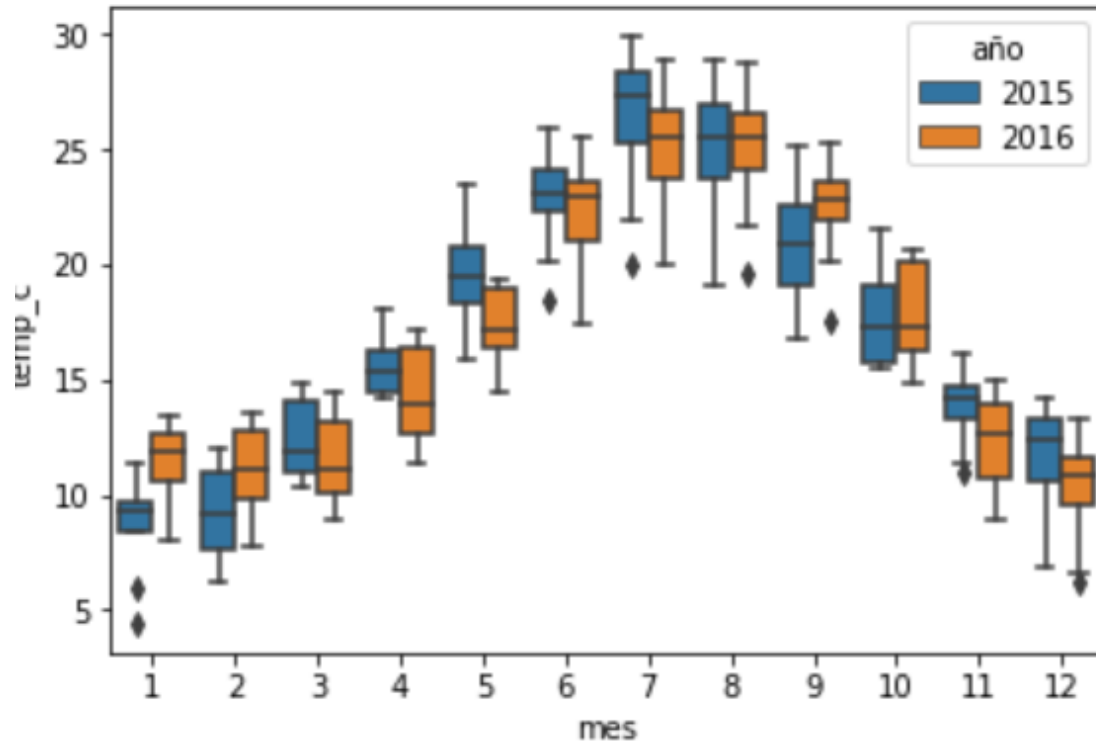
Podemos hacerlo mas complejo **añadiendo un eje x y un eje y**

sns.boxplot (data = “nombre dataframe”, x = “nombre columna”, y = “nombre columna”)



Tambien podemos agrupar los datos haciendo uso del parametro *hue*

```
sns.boxplot ( data = “nombre dataframe”, x = “nombre columna”, y =  
“nombre columna”, hue = “nombre columna”)
```

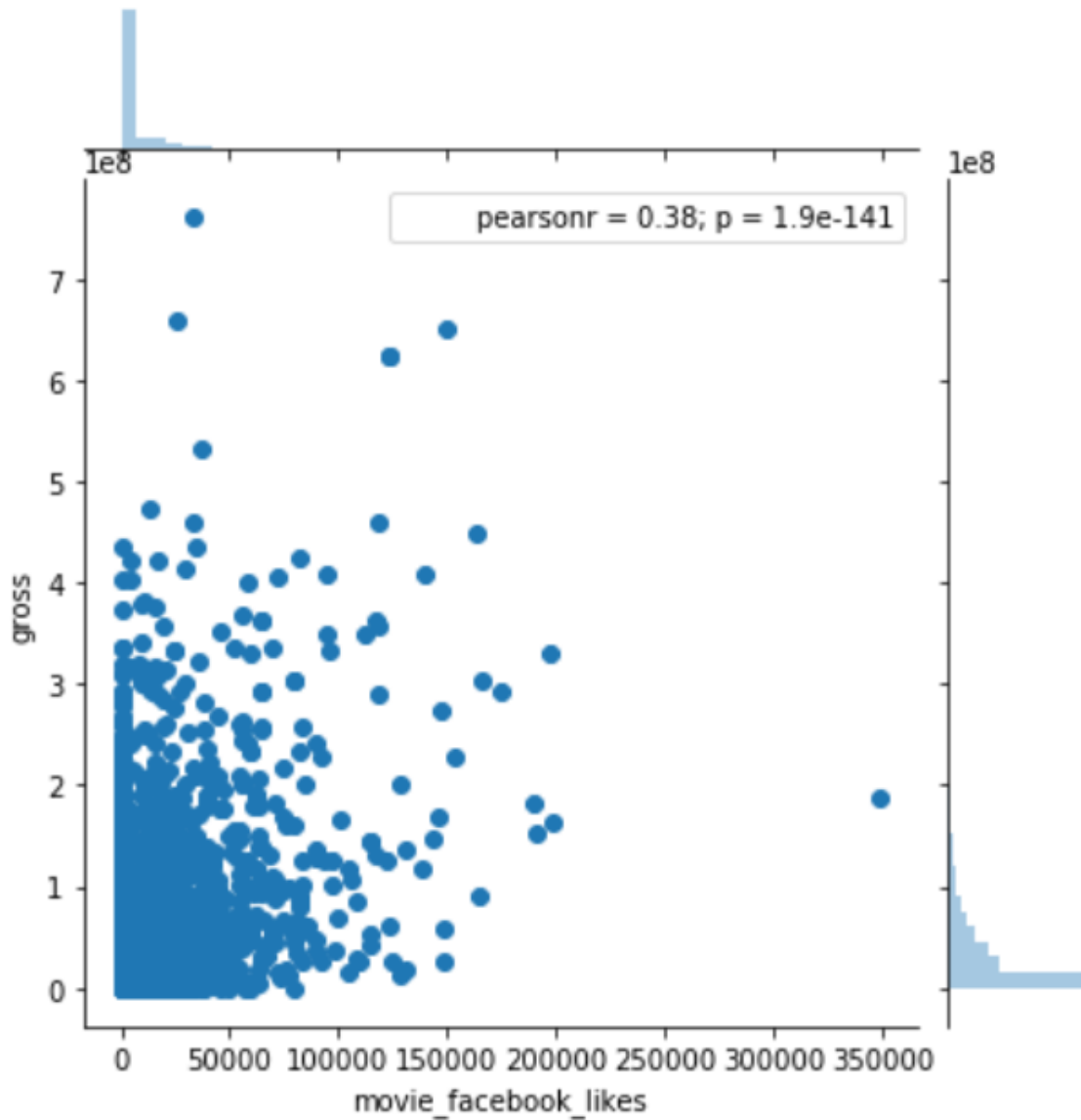


sns.joinplot()

Grafico de relacion Este metodo generara un grafico de distribucion conjunta definida entre 2 variables, junto a los histogramas marginales de cada variable independiente

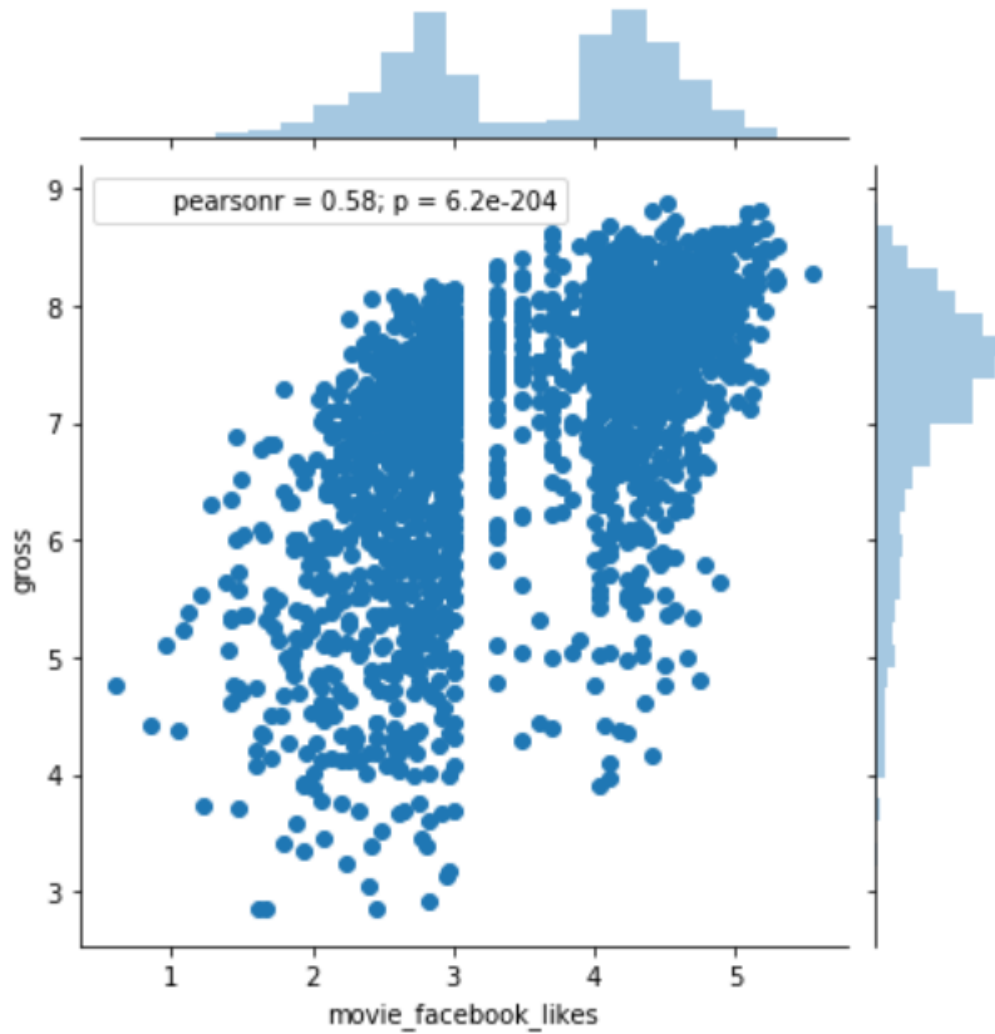
```
sns.joinplot ( data = “nombre dataframe”, x =“nombre columna”, y =  
“nombre columna”)
```

Incluye ademas el coeficiente de pearson, para la correlacion lineal entre las variables

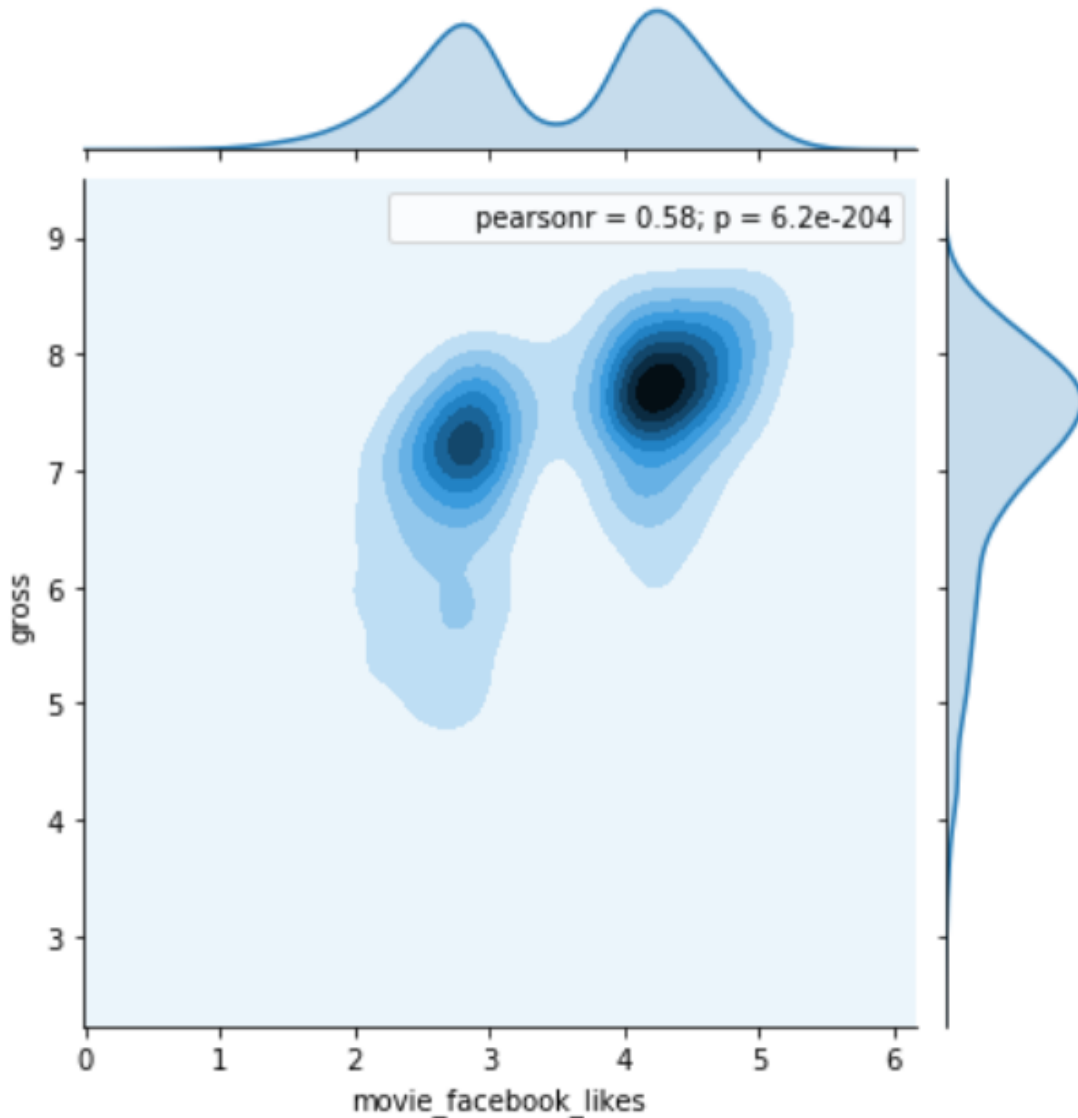


Este metodo incluye mas aspectos a tener en cuenta:

- 1º: Hay veces que al realizarlo nos puede salir **que la disparidad de datos sea muy grande, provocando que el analisis se distorsione**, ante estas situaciones es recomendable ejercer una transformacion de las variables mediante filtros o logaritmos para mejorar la visualizacion.



- 2º: También permiten modificar parámetros en estilo de elemento para mejorar la visualización, **pasando de mapas de puntos de densidad a mapas de densidad de probabilidad estimada**



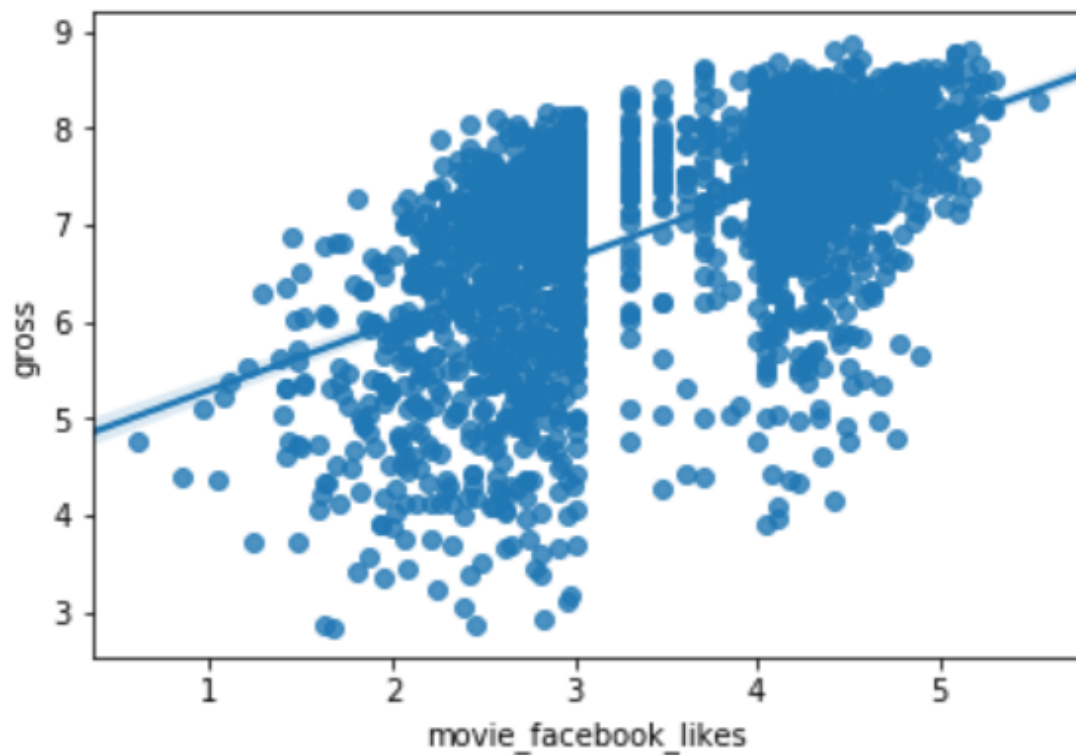
sns.regplot()

Grafico de distribucion Este metodo generara un grafico de distribucion para mostrar puntos para cada observacion mientras muestra un modelo de regresion lineal x / y , pintando la recta al ajuste de intervalo de confianza del 95%

sns.regplot (data = “nombre dataframe”, x = “nombre columna”, y = “nombre columna”)

En este caso podemos evitar que aparezca la recta de regresion lineal, incluyendo el parametro *fit_reg=False*

Ademas si quisieramos realizar una personalizacion mayor de este tipo de grafico, podriamos a traves del metodo *sns.Implot()* que permite un ajuste mayor de parametros, tipo de modelo,...

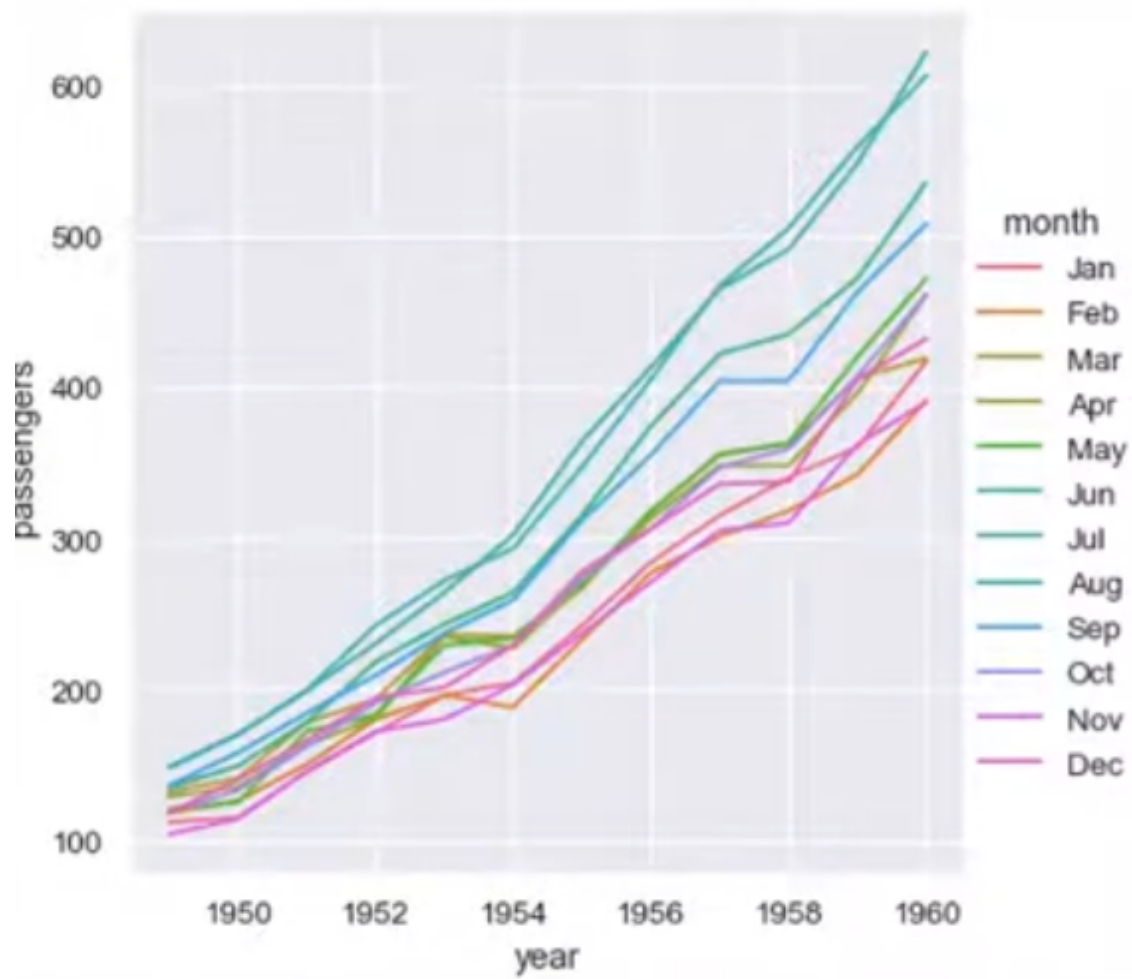


sns.relplot()

Grafico de relacion Este metodo genera un grafico relacional entre 2 variables

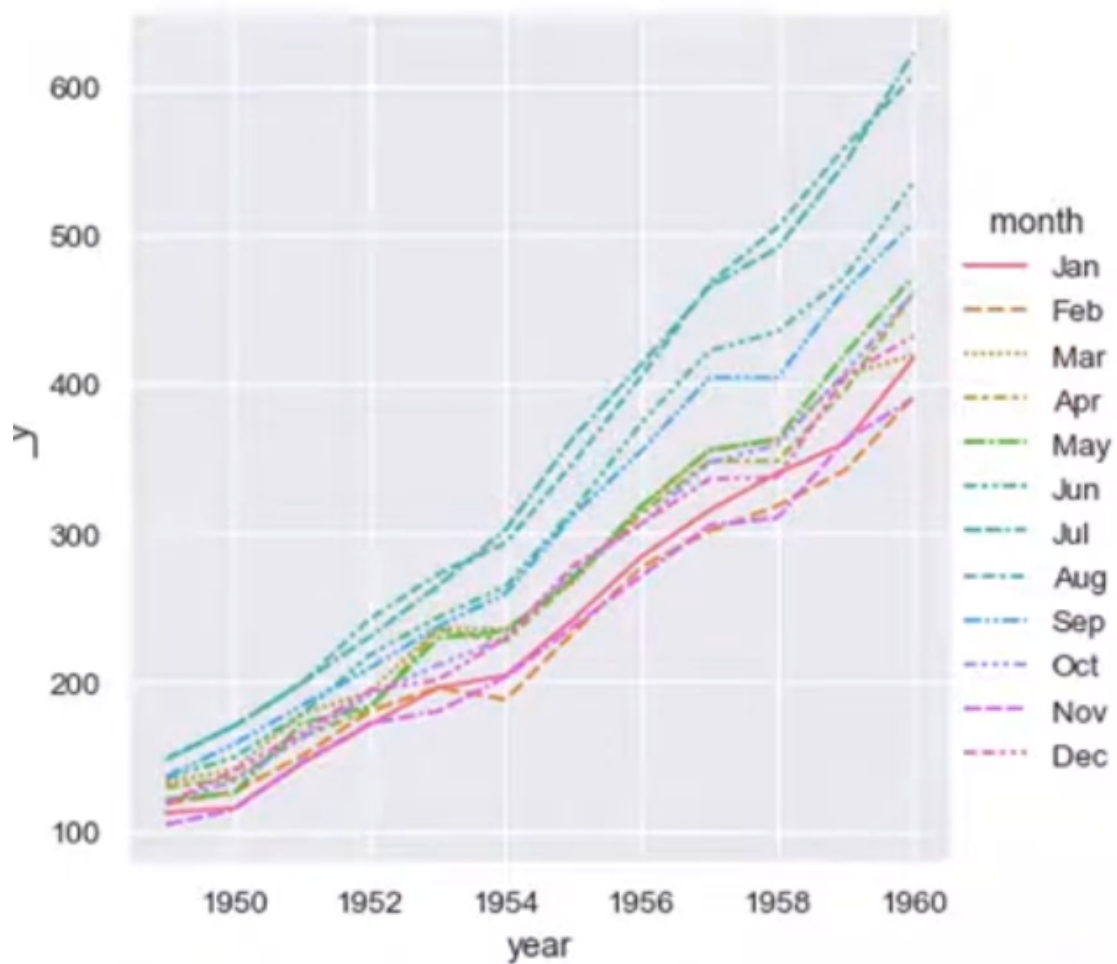
sns.relplot (data = “nombre dataframe” , x = “nombre columna”, y = “nombre columna”, hue = “nombre columna”, kind = “tipo de elemento grafico”)

Permite utilizar parametros para modificar la agrupacion *hue* y la forma del elemento que lo relaciona *kind*



Tambien es posible trabajar con datos *wide-form*:

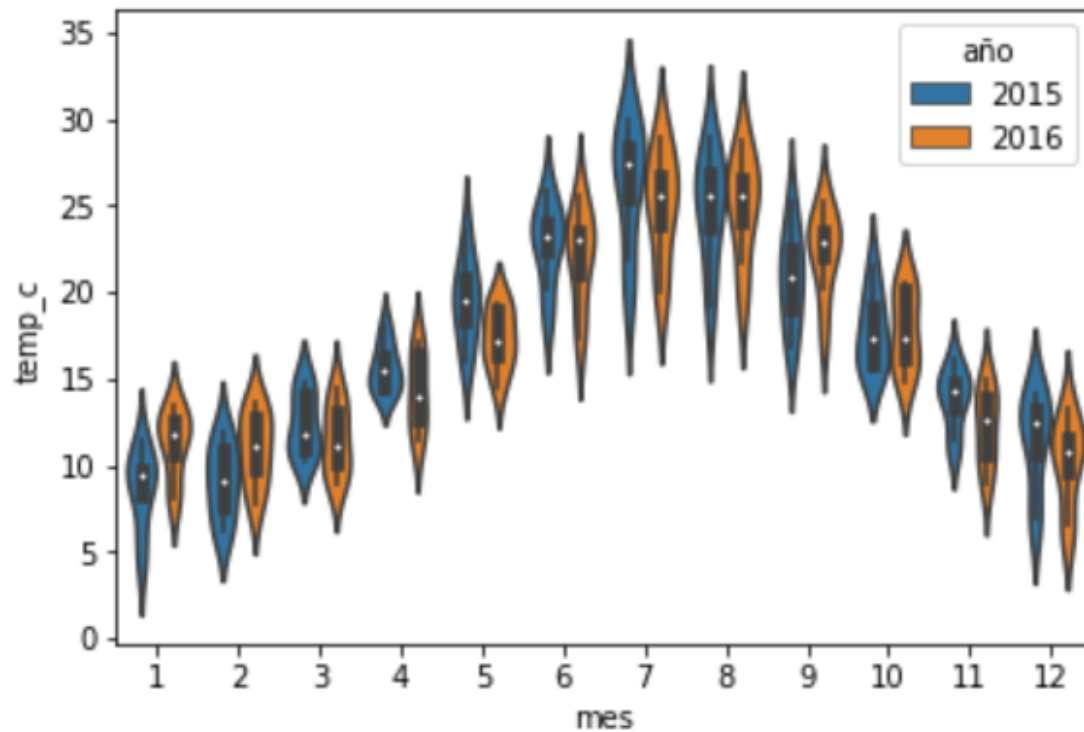
```
***sns.relplot ( data = "nombre dataframe"_wide , kind = "tipo de elemento grafico")***
```



`sns.violinplot()`

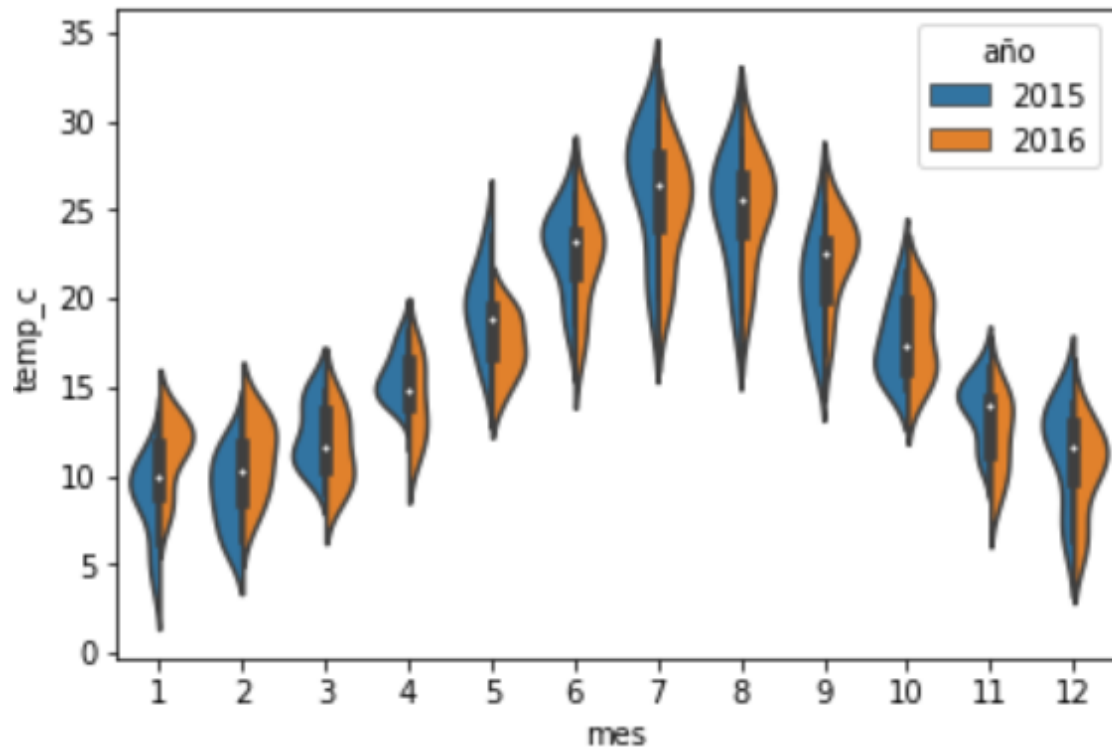
Grafico de categoria Este metodo genera una distribucion de una variable dependiendo de varias categorias y tiene un aspecto de violin. Se busca mostrar una estimacion de la funcion de densidad

`sns.violinplot ( data = "nombre dataframe" , x = "nombre columna" , y = "nombre columna" , hue = "nombre columna" )`



Permite además realizar otra versión en la que **junta los violines** para cada categoría de color, haciendo que cada mitad sea un nivel utilizando el parámetro *split=True*

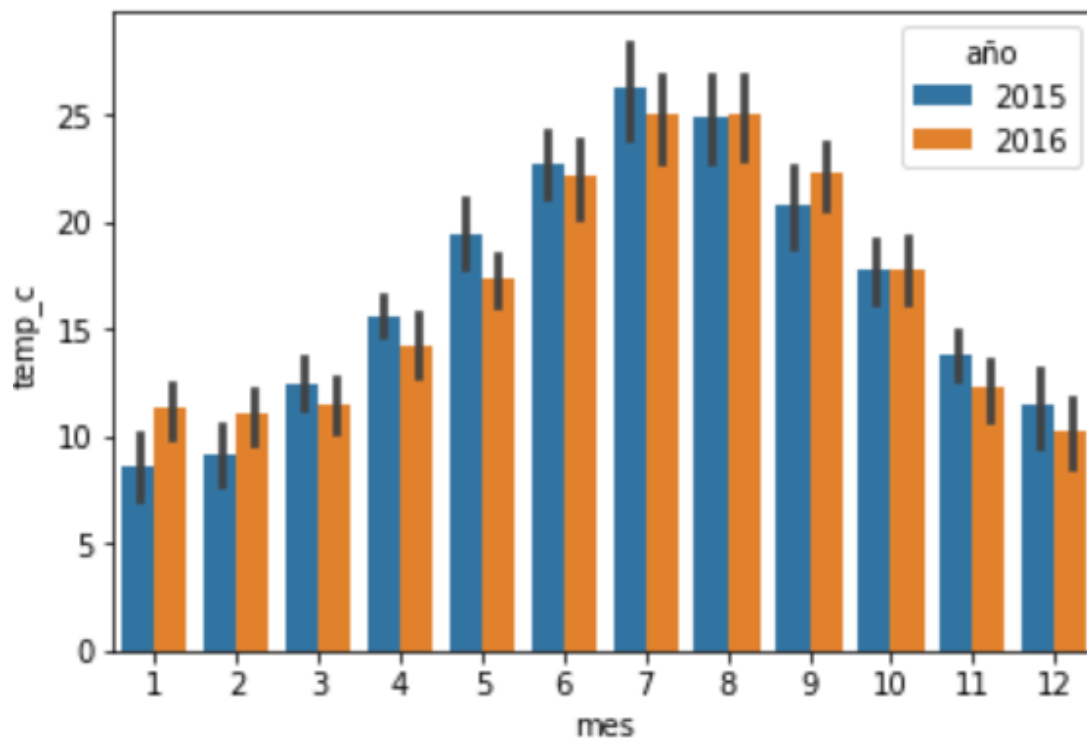
sns.violinplot (data = “nombre dataframe” , x = “nombre columna” , y = “nombre columna” , hue = “nombre columna” , split=True)



sns.barplot()

Grafico de categoria Este metodo genera una distribucion dependiendo de otra categoria y con aspecto de barras. Añade un indicador agregado estadistico de las observaciones para cada categoria (barra)(generalmente es la media)

sns.barplot (data = “nombre dataframe” , x = “nombre columna”, y = “nombre columna”, hue = “nombre columna”)

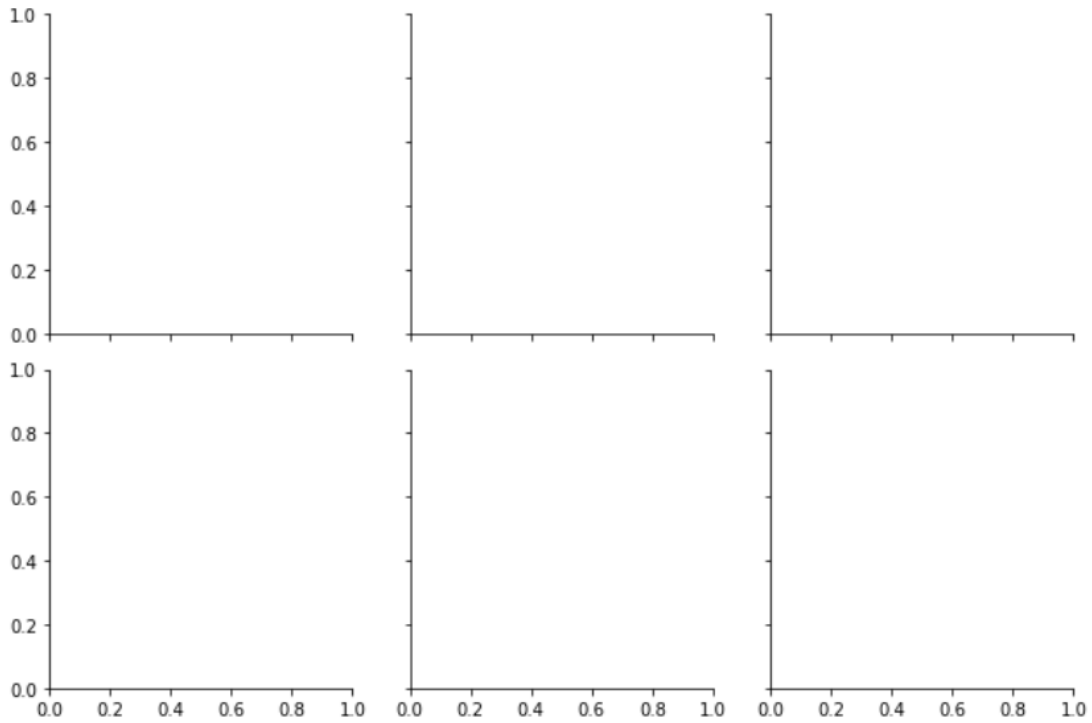


sns.FacetGrid()

Graficos de Facets Este metodo genera una rejilla de datos para utilizar otro metodo que permite crear varios graficos simultaneamente con diferentes vistas de los subconjuntos de datos que hayamos definido.

Primero se define *la rejilla* de datos donde las celdas representan un valor definido por la fila y la columna

sns.FacetGrid (*data* = “nombre dataframe” , *row* = “nombre columna”,
col = “nombre columna”)



En segundo lugar se utiliza otra funcion grafica, que defina el formato de grafico.

sns. "nombre metodo" (data = "nombre dataframe" , x = "nombre columna", y = "nombre columna",...)

